

Representación vectorial del conocimiento (embeddings)

Módulo 2 · Matemáticas y Fundamentos Técnicos de IA

De one-hot a Word2Vec a Transformers: como los embeddings capturan significado y como aplicarlos en búsqueda semántica, clustering y RAG.

1. Introducción · 2. Qué es · 3. Cómo funciona · 4. Ejemplos reales
5. Errores comunes · 6. Aplicación práctica · 7. Relación con módulos · 8. Resumen ejecutivo

Guía de estudio detallada · +2.000 palabras

Representación vectorial del conocimiento: embeddings

La pregunta fundamental del procesamiento del lenguaje natural es: ¿cómo conviertes palabras, frases o documentos en números que un ordenador pueda procesar? La respuesta moderna se llama embedding: una representación vectorial densa en un espacio de alta dimensión donde la proximidad geométrica captura similitud semántica. Es una de las ideas más poderosas y más usadas en la IA aplicada.

Los embeddings son la base de casi todo en IA de lenguaje: la búsqueda semántica en sistemas RAG, la detección de duplicados y plagios, el clustering de tickets de soporte, la recomendación de contenido, la detección de anomalías en texto, y la primera capa de todos los LLMs. Entender cómo se aprenden los embeddings, cómo se usan y cuáles son sus limitaciones es indispensable para diseñar sistemas de IA efectivos.

La evolución de las representaciones textuales

Representaciones clásicas: one-hot y TF-IDF

Las representaciones más antiguas de texto eran sparse (la mayoría de los valores son cero). One-hot encoding asigna a cada palabra un vector de ceros con un único 1 en la posición correspondiente al índice de esa palabra en el vocabulario. Para un vocabulario de 50.000 palabras, cada palabra es un vector de 50.000 dimensiones con 49.999 ceros. El problema: no captura ninguna relación semántica ("perro" y "gato" tienen representaciones completamente ortogonales aunque sean conceptos similares).

TF-IDF (Term Frequency-Inverse Document Frequency) mejora esto ponderando la importancia de cada término: los términos frecuentes en un documento pero raros en el corpus tienen mayor peso. Es el estándar de la recuperación de información basada en palabras clave y sigue siendo útil para búsqueda léxica exacta. Su limitación: no entiende sinónimos (buscar "automóvil" no devuelve documentos sobre "coche") ni polisemia (la palabra "banco" tiene el mismo TF-IDF en contextos financieros y en parques).

Word2Vec y GloVe: el primer salto a representaciones densas

Word2Vec (Google, 2013) fue el primer modelo que aprendió embeddings densos de palabras individuales de forma escalable. La hipótesis distribucional: las palabras que aparecen en contextos similares tienen significados similares. Word2Vec entrena una red neuronal shallow (1 capa oculta) para predecir palabras en el contexto de una palabra central (Skip-Gram) o para predecir una palabra a partir de su contexto (CBOW). Los pesos de la capa oculta resultantes son los embeddings.

La propiedad más famosa de Word2Vec es la aritmética semántica: $\text{vector}(\text{"rey"}) - \text{vector}(\text{"hombre"}) + \text{vector}(\text{"mujer"}) \approx \text{vector}(\text{"reina"})$. Los embeddings capturan relaciones semánticas y sintácticas como analogías geométricas. GloVe (Stanford) logra resultados similares mediante factorización de la matriz de co-ocurrencia de palabras.

La limitación fundamental de Word2Vec y GloVe es que producen embeddings estáticos: cada palabra tiene un único vector independientemente del contexto. La palabra "banco" tiene el mismo embedding en "me senté en el banco del parque" y en "abrí una cuenta en el banco". Esta limitación es superada por los embeddings contextuales.

Embeddings contextuales: BERT y los Transformers

BERT (Google, 2018) introdujo los embeddings contextuales: la representación de cada token depende de todo el contexto de la frase. El mismo token "banco" tiene diferentes embeddings según los tokens que lo rodean. Esto se logra mediante el mecanismo de atención del Transformer: cada token absorbe información de los demás tokens del contexto de forma ponderada.

Sentence Transformers (Reimers y Gurevych, 2019) extienden BERT para producir embeddings de frases completas adecuados para la búsqueda semántica. El modelo all-MiniLM-L6-v2 es el estándar ligero del ecosistema; text-embedding-3-small de OpenAI y text-embedding-ada-002 son los estándares comerciales para sistemas RAG.

SECCIÓN 3 · CÓMO FUNCIONA

El proceso de generar y usar embeddings en producción

El flujo estándar de un sistema de búsqueda semántica con embeddings tiene cuatro pasos. Primero, indexación: cada documento se divide en chunks, cada chunk se convierte en un embedding (1536 dimensiones con text-embedding-3-small), y los embeddings se almacenan en un índice vectorial con metadatos (ID del documento, posición, texto original). Segundo, consulta: la pregunta del usuario se convierte en un embedding con el mismo modelo. Tercero, búsqueda: el índice calcula la similitud coseno entre el embedding de la consulta y todos los embeddings indexados, devolviendo los k más similares. Cuarto, generación: los chunks recuperados se incluyen en el contexto del LLM junto con la pregunta original.

La elección del modelo de embedding tiene un impacto directo en la calidad de la recuperación. Los modelos se evalúan con benchmarks como MTEB (Massive Text Embedding Benchmark), que mide el rendimiento en tareas de recuperación, clasificación y similitud en múltiples idiomas. Para español y otros idiomas europeos, los modelos multilingües (como multilingual-e5-large de Microsoft o nomic-embed-multilingual) suelen superar a los modelos monolingües en inglés.

SECCIÓN 4 · EJEMPLOS REALES

- Clustering de tickets de soporte: generar embeddings de los últimos 10.000 tickets con text-embedding-3-small, reducir a 50 dimensiones con PCA, clustering con k-means (k=20). Los clusters resultantes revelan los 20 tipos principales de problemas que reportan los clientes, información que habría requerido semanas de análisis manual.
- Detección de duplicados en base de conocimiento: una empresa de telecomunicaciones tenía 50.000 artículos en su base de conocimiento interna con mucha redundancia. Generar embeddings de todos los artículos y calcular similitud coseno par a par identificó ~8.000 pares de artículos con similitud > 0.92, candidatos a fusión o eliminación.

- Recomendación de contenido: Spotify usa embeddings de canciones (basados en audio y texto) y de usuarios (basados en historial de escucha) en el mismo espacio semántico. La recomendación es una búsqueda de los k vecinos más cercanos al embedding del usuario en el espacio de embeddings de canciones.
- Semantic search en documentación técnica: el asistente de código de GitHub usa embeddings del repositorio de código y documentación del proyecto para recuperar contexto relevante cuando el desarrollador hace una pregunta. Es RAG sobre código en lugar de texto.

SECCIÓN 5 · ERRORES Y MALENTENDIDOS COMUNES

Error 1: "Los embeddings capturan el significado como un humano." Los embeddings capturan distribuciones estadísticas de co-ocurrencia en el corpus de entrenamiento. Si el corpus tiene sesgos (representaciones negativas de ciertos grupos, ausencia de ciertos dominios de conocimiento), los embeddings reflejan esos sesgos. No hay comprensión semántica genuina.

Error 2: "El mismo modelo de embedding funciona para todos los dominios e idiomas." No. Un modelo de embedding entrenado mayoritariamente en inglés produce embeddings de menor calidad para español, árabe o chino. Un modelo entrenado en texto general puede no representar bien terminología médica, legal o técnica especializada. Evalúa siempre en tu dominio e idioma específicos.

Error 3: "Más dimensiones siempre produce mejor recuperación." No. A partir de cierto punto, las dimensiones adicionales capturan varianza de ruido en lugar de información semántica. La "curse of dimensionality" hace que las distancias coseno en espacios de muy alta dimensión sean menos discriminativas. La dimensionalidad óptima depende del dominio y el tamaño del corpus.

SECCIÓN 6 · APLICACIÓN PRÁCTICA

El proyecto de embedding más accesible y de mayor valor para empezar: toma los últimos 1000-5000 tickets de soporte de tu empresa (o emails de clientes, o documentos internos), genera sus embeddings con text-embedding-3-small de OpenAI, aplica UMAP para reducir a 2 dimensiones y visualiza el espacio semántico con un scatter plot coloreado por categoría o prioridad. Esta visualización, que tarda menos de una hora en implementar, revela la estructura semántica de tu corpus de forma inmediata.

Para sistemas de producción, las consideraciones prácticas son: actualización incremental del índice cuando llegan nuevos documentos (FAISS y la mayoría de las bases vectoriales soportan inserción eficiente sin reindexar todo), gestión de versiones del modelo de embedding (cambiar el modelo requiere reindexar todos los documentos), y monitorización del recall (evalúa periódicamente sobre un conjunto de preguntas con respuestas conocidas que la recuperación devuelva los documentos correctos).

SECCIÓN 7 · RELACIÓN CON OTROS MÓDULOS

- M2-T1 (Álgebra lineal): los embeddings son vectores; la búsqueda semántica es similitud coseno; la reducción de dimensionalidad es PCA/SVD.

- M7 (RAG): los embeddings son la base tecnológica del RAG. Elegir el modelo de embedding correcto es una de las decisiones más importantes en el diseño de un sistema RAG.
- M1-T9 (Atención): los embeddings contextuales de los Transformers son producidos por el mecanismo de atención. Cada token absorbe información del contexto mediante atención.

SECCIÓN 8 · RESUMEN EJECUTIVO

Los embeddings son representaciones vectoriales densas donde la proximidad geométrica captura similitud semántica. La evolución va de one-hot (esparsas, sin semántica) a Word2Vec (densas, estáticas) a BERT/Transformers (densas, contextuales). Los embeddings modernos (text-embedding-3-small, nomic-embed) son la base de la búsqueda semántica, el clustering y el RAG. La elección del modelo de embedding debe basarse en benchmarks específicos del dominio e idioma (MTEB). Los usos más accesibles en empresa: búsqueda semántica, clustering de tickets, detección de duplicados, recomendación de contenido.